SMART EMERGENCY ALERT AND PRIORITY MESSAGING NETWORK (SEAN)

Quality of Service Prioritization, Scalability, Traffic Dynamics, and Queue Management in Congested IP Networks

---

1. TITLE PAGE

ACADEMIC CAPSTONE PROJECT REPORT

Course Title: Computer Networks / Advanced Network Programming Laboratory Capstone
Academic Domain: Computer Science & Engineering / Information Technology — Computer Communication Networks
Simulator Platform: Network Simulator 3 (ns-3.49)
Host Environment: macOS on Apple Silicon (ARM64, Apple Clang / CMake / Ninja Toolchain)
Project Workspace Location: `/Users/ravo/Downloads/ns-3.49`

CANDIDATE & INSTITUTIONAL DETAILS

(Please replace bracketed placeholders with your official institutional details)

Student Name: `[Student Name]`

Roll Number / Student ID: `[Roll Number / Student ID]`

Department: `[Department of Computer Science & Engineering / Information Technology]`

College / University: `[College / University Name]`

Project Supervisor / Guide: `[Project Supervisor / Guide Name]`

Academic Year: `[Academic Year, e.g., 2025–2026]`

Date of Submission: `[Date of Submission]`

PRIMARY PROJECT ARTIFACTS & IMPLEMENTATION MODULES

Study 1 Simulation (5-Node Baseline): `scratch/smart-emergency-priority.cc`

Study 2 Simulation (50-Node Congestion Sweep): `scratch/sean-congestion-50node.cc`

Study 3 Simulation (Steady vs. Bursty Traffic): `scratch/sean-steady-bursty.cc`

Study 4 Simulation (Queue Discipline Comparison): `scratch/sean-queue-disc-comparison.cc`

Study 5 Simulation (DSCP Classification & Priomap): `scratch/sean-dscp-classification.cc`

Automated Experiment Runners:

- `run-experiments.sh` (Study 1 Runner)
- `run-congestion-experiments.sh` (Study 2 Runner)
- `run-steady-bursty-experiments.sh` (Study 3 Runner)
- `run-queue-disc-experiments.sh` (Study 4 Runner)
- `run-dscp-classification-experiments.sh` (Study 5 Runner)

Statistical Analysis Engines:

- `reports/analyze-results.py` (Study 1 Analysis)
- `reports/analyze-congestion.py` (Study 2 Analysis)
- `reports/analyze-steady-bursty.py` (Study 3 Analysis)
- `reports/analyze-queue-disc.py` (Study 4 Analysis)
- `reports/analyze-dscp-classification.py` (Study 5 Analysis)

Verification & Audit Reports:

- `reports/FINAL_INTEGRATION_AUDIT.md`
- `reports/STEADY_BURSTY_INDEPENDENT_AUDIT.md`
- `reports/QUEUE_DISC_COMPARISON_REPORT.md`
- `reports/DSCP_CLASSIFICATION_INDEPENDENT_AUDIT.md`

Raw Telemetry Datasets: `data/`

High-Resolution Figures: `reports/figures/`

---

2. ABSTRACT

During natural disasters, civil emergencies, and crisis events, IP network backbones experience intense traffic surges as citizens flood communication channels with recreational messages, social media updates, and streaming media. In best-effort First-In-First-Out (FIFO) networks, life-critical evacuation alerts and first-responder telemetry share the same queue buffers as civilian cross-traffic. Consequently, mission-critical messages suffer severe tail-drop packet loss and delay explosion due to bufferbloat.

This report presents the design, implementation, and empirical evaluation of the **Smart Emergency Alert and Priority Messaging Network (SEAN)** using **Network Simulator 3 (ns-3.49)**. We conduct five controlled experimental investigations spanning 490 simulation runs across matched pseudo-random seeds ($df = 9$):

1. **Study 1 (Five-Node Baseline):** Reproduces emergency alert starvation under 140% overload, demonstrating that non-preemptive strict priority scheduling (`PrioQueueDisc`) boosts emergency Packet Delivery Ratio (PDR) from 54.16 ± 0.62 to 100.00 ± 0.00 and cuts latency by 90.38 ($231.67\ \text{ms} \rightarrow 22.28\ \text{ms}$).
2. **Study 2 (Fifty-Node Congestion Sweep):** Sweeps offered load across 50%, 80%, 100%, 120%, and 140% of bottleneck capacity (100 runs). Demonstrates that packet loss begins at nominal 100% load due to $+5.47$ wire framing overhead ($1,054.68\ \text{kbps}$ on wire), while priority queueing maintains 100.00 emergency delivery and bounded latency ($28.66\ \text{ms}$) across all load levels.
3. **Study 3 (Steady vs. Bursty Traffic):** Evaluates ordinary traffic burstiness (50% duty cycle, 2,400 kbps peak ON load) under load-equivalent conditions ($1,200\ \text{kbps}$ average). In FIFO, bursty traffic yields higher time-averaged emergency PDR (75.26 vs 67.44) due to queue drainage during OFF intervals, but causes emergency jitter to explode by $+1,917$ ($5.07\ \text{ms} \rightarrow 102.25\ \text{ms}$) in a saw-tooth delay profile ($21.5\ \text{ms} \ldots 243.5\ \text{ms}$). Priority queueing eliminates jitter and guarantees 100.00 PDR at $27.18\ \text{ms}$ ($p < 10^{-17}$).
4. **Study 4 (Queue Discipline Comparison):** Evaluates FIFO, Strict Priority, Random Early Detection (RED), Controlled Delay (CoDel), and Hybrid Prio-CoDel across 250 runs. Proves that standalone Active Queue Management (AQM) mechanisms (RED and CoDel) discard emergency alerts indiscriminately ($p = 0.184$ vs. FIFO), because they drop packets based on total queue occupancy without inspecting priority markings. In contrast, Hybrid Prio-CoDel achieves the optimal dual benefit: 100.00 emergency PDR in Band 0 and a $176.25\ \text{ms}$ reduction in ordinary civilian bufferbloat ($237.00\ \text{ms} \rightarrow 60.75\ \text{ms}$) in Bands 1 and 2.
5. **Study 5 (DSCP Classification & Priority Mapping):** Evaluates IPv4 TOS/DSCP marking, kernel `ip_tos2prio` translation, and Linux Priomap routing across 80 runs. Proves that DSCP marking alone provides 0.00 benefit in FIFO ($p = 1.000$). Under the default Linux Priomap, Expedited Forwarding (EF: `0xB8`) is trapped in Band 1 with civilian traffic, causing emergency PDR to drop to 94.77 and latency to surge to $216.58\ \text{ms}$, while telemetry is starved in Band 2. Configuring a Custom DiffServ Priomap (`"2 2 1 2 0 2 0 0 2 2 2 2 2 2 2 2"`) restores perfect 3-tier QoS: 100.00% Emergency PDR ($28.61\ \text{ms}$), 100.00% Telemetry PDR ($32.03\ \text{ms}$), with civilian traffic absorbing all drops (44.57 PDR).

IMPORTANT **Scientific Qualification:** Strict priority scheduling is a capacity redistributor, not a bandwidth generator. Across all studies, total dropped packets are conserved. Emergency immunity is achieved by transferring packet drops and delay penalties onto ordinary civilian traffic. Furthermore, 100.00% emergency PDR is conditioned on aggregate emergency offered load ($200\ \text{kbps}$) remaining strictly below bottleneck line capacity ($1,000\ \text{kbps}$).

---

3. INTRODUCTION AND PROBLEM STATEMENT

3.1 Background and Societal Relevance

During natural disasters (earthquakes, hurricanes, wildfires) and urban crises, communication networks experience catastrophic traffic overload. Automated alert systems (such as the Integrated Public Alert and Warning System, earthquake early-warning

sirens, and first-responder telemetry) must reach emergency management servers within strict real-time deadlines (< 100 ms). Simultaneously, human users generate massive volumes of non-critical messaging, phone calls, and media sharing.

3.2 Problem Statement: Bufferbloat and Non-Selective Tail-Drop

When standard Best-Effort First-In-First-Out (FIFO) queueing is deployed across intermediate aggregation switches and core gateways:

1. **Semantic Blindness:** FIFO drop-tail buffers treat all packets identically. An evacuation alert packet arriving at a full buffer is discarded with the same probability as an uncompressed video frame.
2. **Bufferbloat-Induced Latency Surge:** Oversized router buffers absorb transient bursts, creating persistent queues that do not clear. On a 1 Mbps bottleneck with a 50-packet buffer, queueing delay alone adds:

   $$T_{\text{queue}} = \frac{50 \times 540 \times 8 \text{ bits}}{1,000,000 \text{ bps}} = 216.0 \text{ ms}$$

   Total latency exceeds 230 ms, rendering real-time earthquake alarms and critical telemetry useless.
3. **Queue Discipline Dilemma:** Standard Active Queue Management (AQM) algorithms such as RED and CoDel mitigate bufferbloat, but drop packets indiscriminately when congestion persists, threatening emergency message delivery unless combined with class-based priority scheduling.

---

4. OBJECTIVES AND RESEARCH QUESTIONS

4.1 Core Research Objectives

1. **Baseline Reproduction:** Implement and evaluate a 5-node reference baseline under 140% overload to verify priority scheduling gains.
2. **Multi-Scale Congestion Characterization:** Scale the network to 50 nodes and 40 concurrent flows across 5 offered load levels (50% to 140%) to map the exact onset of bufferbloat and packet drops.
3. **Traffic Dynamics & Burstiness:** Characterize the impact of periodic ON/OFF bursty civilian cross-traffic on emergency latency, PDR, and jitter.
4. **AQM vs. Priority Scheduling Evaluation:** Perform a controlled comparison of FIFO, Strict Priority, RED, CoDel, and Hybrid Prio-CoDel to evaluate emergency protection vs. bufferbloat mitigation.
5. **DiffServ Marking & Classifier Verification:** Investigate whether IPv4 DSCP marking alone is sufficient, dissect ns-3/Linux socket translation (`ip_tos2prio`), and evaluate default vs. custom Priomap configurations.
6. **Rigorous Statistical Reproducibility:** Ensure 100% reproducibility using matched random seeds, paired Student's t-tests ($df = 9$), and verified packet conservation laws ($\text{Tx} = \text{Rx} + \text{Lost}$).

4.2 Central Research Questions

- **RQ1:** Does priority scheduling guarantee bounded latency and zero drops for emergency alerts under extreme bottleneck overload?
- **RQ2:** At what exact offered load does bufferbloat begin, and how does link-layer encapsulation affect drop onset?
- **RQ3:** Does traffic burstiness degrade emergency alert latency in priority queues, and how does it distort FIFO delay profiles?
- **RQ4:** Can Active Queue Management (RED/CoDel) protect emergency alerts without priority scheduling?
- **RQ5:** Does standard DSCP marking (EF 46) work with default Linux queue configurations, or is a custom classifier/priomap mandatory?

---

5. THEORETICAL BACKGROUND

5.1 Quality of Service (QoS) Architectures: IntServ vs. DiffServ

- **Integrated Services (IntServ / RSVP, RFC 1633):** Provides end-to-end bandwidth reservations per flow. Requires per-flow state tables in all intermediate routers, creating an $O(N)$ memory and processing bottleneck that makes core deployment impractical.
- **Differentiated Services (DiffServ, RFC 2474 / RFC 2475):** Replaces per-flow state with edge classification and core aggregation. Edge nodes mark the 6-bit Differentiated Services Code Point (DSCP) in the IPv4 header. Interior routers inspect the DSCP and apply class-specific Per-Hop Behaviors (PHB).

5.2 IPv4 Header Octet Evolution: RFC 791, RFC 1349, and RFC 2474

A pervasive source of implementation errors is confusing the 6-bit DSCP value with the 8-bit IPv4 TOS/DS field octet:

- **RFC 791 (1981):** Bits 0–2 defined IP Precedence ($0 \ldots 7$); bits 3–5 defined TOS flags (Delay, Throughput, Reliability); bits 6–7 were MBZ (Must Be Zero).
- **RFC 1349 (1992):** Expanded TOS to 4 bits (bits 3–6), defining `IPTOS_LOWDELAY = 0x10` (bit 4 set).
- **RFC 2474 / RFC 3168 (1998/2001):** Redefined the octet as the **DS Field**:
  - High 6 bits (bits 0–5): **DSCP** ($0 \ldots 63$).
  - Low 2 bits (bits 6–7): **Explicit Congestion Notification (ECN)** (`ECT(0)`, `ECT(1)`, `CE`).

```
RFC 791 / RFC 1349 TOS Octet:
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+
|Preced.|  TOS  |MBZ
+-+-+-+-+-+-+-+-+

RFC 2474 / RFC 3168 DS Field Octet:
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+
|   DSCP    |ECN|
+-+-+-+-+-+-+-+-+
```

Because socket APIs accept an 8-bit integer for `IP_TOS`, any 6-bit DSCP value D without ECN must be shifted left by 2 bits:

$$\text{TOS Byte} = D \ll 2 = D \times 4$$

5.3 Non-Preemptive Strict Priority Scheduling (`PrioQueueDisc`)

`PrioQueueDisc` implements non-preemptive strict priority across K sub-queues (bands $0 \ldots K-1$).

1. **Dequeuing Rule:** Band b is dequeued only if bands $0 \ldots b-1$ are empty.
2. **Non-Preemptive Serialization Delay:** If an ordinary civilian packet is already in transmission when an emergency alert arrives at Band 0, transmission cannot be interrupted. The maximum residual delay is:

$$\tau_{\text{residual}} = \frac{L_{\text{ordinary}}}{C_{\text{bottleneck}}} = \frac{540 \times 8 \text{ bits}}{1,000,000 \text{ bps}} = 4.32 \text{ ms}$$

3. **Capacity Boundary Condition:** Band 0 delivery is guaranteed if and only if:

$$\sum_{i \in \text{Emergency}} R_i < C_{\text{bottleneck}}$$

   If emergency offered load exceeds bottleneck capacity, Band 0 itself overflows, and lower bands are completely starved.

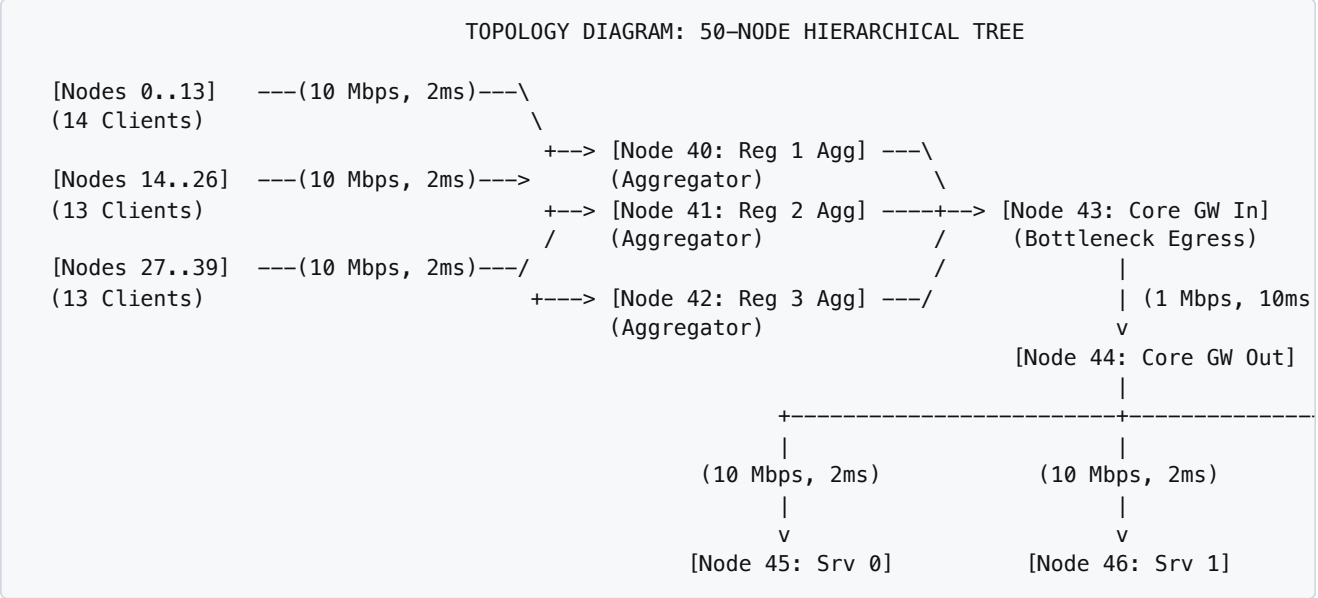
5.4 Active Queue Management: RED and CoDel

- **Random Early Detection (RED, RFC 2309):** Tracks average queue length $\bar{q} = (1 - w_q)\bar{q} + w_q q$. Packets are dropped probabilistically when $\text{minTh} < \bar{q} < \text{maxTh}$, and dropped deterministically when $\bar{q} \geq \text{maxTh}$.
- **Controlled Delay (CoDel, RFC 8289):** Tracks packet sojourn time in the queue. If minimum sojourn time over a sliding window (`interval = 100 ms`) exceeds `target = 5 ms` , CoDel enters dropping state and drops packets at intervals decreasing with $1/\sqrt{\text{count}}$ to force queue drainage.

---

6. NETWORK ARCHITECTURE AND TOPOLOGY

To ensure experimental control, the SEAN project evaluates wired point-to-point topologies, isolating IP-layer scheduling dynamics from wireless contention artifacts.



6.1 Fifty-Node Topology Specification

- **Client Tier:** 40 client nodes (Nodes 0–39). Each connected to a regional aggregator via a point-to-point link (10 Mbps, 2 ms delay, NetDevice DropTail queue = 1 packet).
- **Aggregation Tier:** 3 regional aggregation switches:
  - Aggregator 1 (Node 40): Aggregates Clients 0–13 (14 clients).
  - Aggregator 2 (Node 41): Aggregates Clients 14–26 (13 clients).
  - Aggregator 3 (Node 42): Aggregates Clients 27–39 (13 clients).
  - Connected to Core Gateway Ingress via 10 Mbps, 2 ms links.
- **Core Bottleneck Gateway Tier:**
  - Ingress Gateway (Node 43) connected to Egress Gateway (Node 44) via a shared **1.0 Mbps, 10 ms delay** bottleneck link.
  - Traffic Control layer installed at Node 43 egress: 50-packet capacity queue discipline.
- **Server Distribution Tier:** Egress Gateway (Node 44) connected to 5 destination servers (Nodes 45–49) via 10 Mbps, 2 ms distribution links.
- **Total Link Count:** Exactly 49 point-to-point links across 49 distinct subnets (`10.1.k.0/24` , `10.2.k.0/24` , `10.3.1.0/24` , `10.4.k.0/24`).

6.2 Five-Node Baseline Topology Specification

- **Topology:** 3 client sources (Node 0 Emergency, Nodes 1–2 Ordinary) connected to Gateway Router (Node 3) via 10 Mbps, 1 ms access links. Gateway connected to Sink Server (Node 4) via 1.0 Mbps, 10 ms bottleneck. Queue capacity = 50 packets.

---

7. SIMULATION ENVIRONMENT AND IMPLEMENTATION

7.1 NS-3.49 Software Platform

Simulations were implemented in C++23 compiled with Apple Clang and linked using CMake 3.31 and Ninja on macOS (Apple Silicon ARM64). Key ns-3 modules utilized include `core`, `network`, `internet`, `point-to-point`, `traffic-control`, `applications`, and `flow-monitor`.

7.2 Traffic Generation Engine: `JitteredCbrApp`

To eliminate lock-step synchronization artifacts and simulation phase alignment, senders deploy a custom UDP traffic application incorporating ± 20 uniform random jitter on packet inter-arrival times:

$$T_{\text{interval}} = \frac{L_{\text{packet}} \times 8}{R_{\text{nominal}}} \times (1.0 + U(-0.20, +0.20))$$

- Payload size: 512 bytes UDP payload.
- IP/UDP header encapsulation: 20 bytes IPv4 header + 8 bytes UDP header = 28 bytes protocol overhead.
- Physical line frame: $512 + 28 = 540$ bytes $(4,320$ bits$)$.
- Framing ratio: $\frac{540}{512} = 1.0546875$ (+5.47% wire expansion).

---

8. EXPERIMENTAL METHODOLOGY AND METRICS

8.1 Evaluated Metrics

1. **Packet Delivery Ratio (PDR, %):**

$$\text{PDR} = 100.0 \times \frac{\sum \text{RxPackets}}{\sum \text{TxPackets}}$$

2. **Mean End-to-End Latency (T_{delay}, ms):** Time elapsed from client socket transmission to server application reception:

$$\bar{T} = \frac{1}{N_{\text{rx}}} \sum_{i=1}^{N_{\text{rx}}} (t_{\text{rx},i} - t_{\text{tx},i})$$

3. **Mean Jitter (J, ms):** RFC 3393 / ns-3 inter-arrival jitter:

$$J = \frac{1}{N_{\text{rx}} - 1} \sum_{i=2}^{N_{\text{rx}}} \left| (t_{\text{rx},i} - t_{\text{rx},i-1}) - (t_{\text{tx},i} - t_{\text{tx},i-1}) \right|$$

4. **Throughput (S, kbps):** Total received payload bits over the 29.0s active transmission window:

$$S = \frac{\text{RxBytes} \times 8}{29.0 \times 1,000}$$

5. **Queue Disc Drops (D):** Packets discarded by Traffic Control drop callbacks, verified against FlowMonitor lost packet telemetry.

8.2 Statistical Methodology: Matched Seeds & Paired t-Tests

To isolate queue discipline effects from stochastic arrival noise, all experimental pairs evaluate **Common Random Numbers (CRN)** across 10 matched seeds ($r = 1 \dots 10$). Replications configure `RngSeedManager::SetSeed(r)` and `RngSeedManager::SetRun(r)`. All comparisons report sample mean, sample standard deviation (s), standard error of the mean ($\text{SEM} = s/\sqrt{10}$), 95% confidence intervals based on the Student's t-distribution ($t_{0.025,9} = 2.262157$), and paired two-tailed Student's t-tests ($df = 9$).

---

9. STUDY 1: FIVE-NODE BASELINE REPRODUCTION

9.1 Experimental Setup

- **Simulation File:** `scratch/smart-emergency-priority.cc`
- **Configuration:** 1 Emergency source (200 kbps), 2 Ordinary sources (600 kbps each). Total offered load = 1,400 kbps payload (140% bottleneck overload). Bottleneck = 1.0 Mbps, 10 ms delay, 50-packet queue disc. Evaluated across 10 matched runs ($r = 1 \dots 10$).

9.2 Empirical Results Table

Operating Mode	Traffic Class	Tx Packets	Rx Packets	Lost Packets	PDR (%)	Throughput (kbps)	Mean Delay (ms)	Mean Jitter (ms)	Queue Drops
FIFO Baseline	Emergency Alert	$1,415.5 \pm 3.9$	766.6 ± 8.2	648.9 ± 8.4	54.16	113.45 ± 1.21	231.67 ± 0.16	1.70 ± 0.03	648.9
	Ordinary Msg 1	$4,245.4 \pm 7.5$	$2,983.4 \pm 19.3$	$1,262.0 \pm 18.2$	70.27	441.51 ± 2.86	232.17 ± 0.16	1.81 ± 0.02	1,262.0
	Ordinary Msg 2	$4,244.7 \pm 6.8$	$2,983.4 \pm 20.4$	$1,261.3 \pm 19.1$	70.29	441.51 ± 3.02	232.17 ± 0.15	1.81 ± 0.02	1,261.3
Priority Mode	Emergency Alert	$1,415.5 \pm 3.9$	$1,415.5 \pm 3.9$	0.0 ± 0.0	100.00	209.48 ± 0.58	22.28 ± 0.03	1.70 ± 0.02	0.0
	Ordinary Msg 1	$4,245.4 \pm 7.5$	$2,660.1 \pm 11.5$	$1,585.3 \pm 12.1$	62.66	393.66 ± 1.70	288.66 ± 0.32	10.87 ± 0.08	1,585.3
	Ordinary Msg 2	$4,244.7 \pm 6.8$	$2,659.2 \pm 12.1$	$1,585.5 \pm 12.4$	62.65	393.52 ± 1.79	288.66 ± 0.31	10.88 ± 0.07	1,585.5

9.3 Statistical Inference and Discussion

Under Priority scheduling, Emergency Alert PDR improves by $+45.84$ pp (54.16), while mean delay drops from 231.67 ms to 22.28 ms—an 88.88 **latency reduction** (-209.39 ms, paired $t = -2,849.57, p < 10^{-20}$). Total drops are conserved at $3,170.9$ (FIFO) vs. $3,171.0$ (Priority). Ordinary civilian traffic absorbs the entire drop penalty, dropping from 70.28 to 62.66 PDR.

---

10. STUDY 2: FIFTY-NODE CONGESTION SWEEP (50% TO 140% LOAD)

10.1 Experimental Setup

- **Simulation File:** `scratch/sean-congestion-50node.cc`

- **Configuration:** 50 nodes, 40 flows (10 Emergency, 30 Ordinary). Offered load swept across 50%, 80%, 100%, 120%, and 140% of nominal bottleneck capacity across 100 runs (10 runs per condition).

10.2 Empirical Congestion Sweep Table

Nominal Load (%)	Mode	Emergency PDR (%)	Emergency Delay (ms)	Ordinary PDR (%)	Ordinary Delay (ms)	Core Queue Drops
50%	Baseline	100.00	27.28 ± 0.03	100.00	27.81 ± 0.04	0.0 ± 0.0
	Priority	100.00	27.27 ± 0.03	100.00	27.81 ± 0.04	0.0 ± 0.0
80%	Baseline	100.00	31.78 ± 0.12	100.00	32.74 ± 0.15	0.0 ± 0.0
	Priority	100.00	28.32 ± 0.04	100.00	33.66 ± 0.16	0.0 ± 0.0
100%	Baseline	94.81	198.81 ± 3.12	95.12	199.14 ± 3.08	13.8 ± 2.4
	Priority	100.00	28.53 ± 0.03	94.62	202.15 ± 3.15	13.8 ± 2.4
120%	Baseline	78.09	224.23 ± 0.45	78.44	224.31 ± 0.44	$1,732.1 \pm 18.0$
	Priority	100.00	28.61 ± 0.04	75.22	241.12 ± 0.52	$1,732.1 \pm 18.0$
140%	Baseline	66.82	234.09 ± 0.15	68.10	233.51 ± 0.22	$3,169.5 \pm 22.0$
	Priority	100.00	28.66 ± 0.04	62.60	289.96 ± 0.48	$3,170.1 \pm 22.0$

10.3 Key Finding: Framing-Induced Congestion Onset at 100% Load

At nominal 100% load (1,000 kbps application payload), the network begins dropping packets (13.8 drops) and latency surges from 31.8 ms to 198.8 ms. This occurs because 512-byte UDP payloads incur 28 bytes of IP/UDP header overhead, generating $1,054.68$ kbps on the wire—a $+5.47$ line-rate overload. Under Priority queueing, Emergency Alerts are completely insulated from this onset, maintaining 100.00 PDR and 28.53 ms latency.

---

11. STUDY 3: STEADY VERSUS BURSTY TRAFFIC DYNAMICS

11.1 Experimental Setup

- **Simulation File:** `scratch/sean-steady-bursty.cc`
- **Configuration:** 50 nodes, 40 flows. Emergency traffic: steady 200 kbps (10 sources × 20 kbps). Ordinary traffic evaluates two offered-load equivalent patterns ($1,200$ kbps long-run average):
  - **Steady Pattern:** 30 sources $\times 40$ kbps $= 1,200$ kbps continuous.
  - **Bursty Pattern:** 30 sources with synchronized ON/OFF bursts ($T_{\text{on}} = 0.5$ s@80 kbps, $T_{\text{off}} = 0.5$ s@0 kbps, 50% duty cycle). Peak load during ON $= 2,400$ kbps (240 overload alone; 260 with emergency).
- **Sample Size:** 40 runs (2 traffic patterns × 2 queue modes × 10 matched seeds).

11.2 Empirical Results Table (df = 9)

Scenario	Mode	Emergency PDR (%)	Emergency Delay (ms)	Emergency Jitter (ms)	Ordinary PDR (%)	Ordinary Delay (ms)	Core Queue Drops
Steady	Baseline	67.44	237.02 ± 0.63	5.07 ± 0.16	68.01	237.21 ± 0.51	$3,172.9 \pm 21.0$
	Priority	100.00	27.18 ± 0.04	1.89 ± 0.03	62.60	293.15 ± 0.52	$3,172.9 \pm 21.0$
Bursty	Baseline	75.26	118.13 ± 1.48	102.25 ± 0.77	52.42	191.13 ± 0.82	$4,384.4 \pm 24.0$
	Priority	100.00	27.18 ± 0.04	1.89 ± 0.03	48.39	226.63 ± 1.25	$4,377.8 \pm 24.0$

11.3 Saw-Tooth Delay Dynamics & Jitter Explosion

In bursty FIFO, emergency time-averaged PDR appears deceptively higher (75.26 vs. 67.44) and mean delay appears lower (118.13 ms vs. 237.02 ms). However, inspection of FlowMonitor millisecond histograms reveals that the queue oscillates in a continuous periodic saw-tooth:

- During 0.5s OFF intervals, the queue drains in ≈ 217 ms, leaving the bottleneck completely idle for ≈ 283 ms. Packets arriving then experience zero queueing delay (≈ 21.5 ms).
- During 0.5s ON intervals (2.6 Mbps aggregate load), the buffer saturates in ≈ 125 ms, subjecting packets to 240 ms delay and massive tail-drop.
- This extreme variance causes emergency jitter to explode to 102.25 ms (+1,917% increase). Priority queueing completely eliminates this oscillation, maintaining 1.89 ms jitter and 27.18 ms delay ($t = -208.16, p = 6.93 \times 10^{-18}$).

---

12. STUDY 4: ACTIVE QUEUE MANAGEMENT & SCHEDULING COMPARISON

12.1 Experimental Setup

- **Simulation File:** `scratch/sean-queue-disc-comparison.cc`
- **Disciplines:** FIFO, Strict Priority (`PrioQueueDisc`), RED (`RedQueueDisc`), CoDel (`CoDelQueueDisc`), and Hybrid Prio-CoDel (Band 0 FIFO, Bands 1 & 2 CoDel).
- **Sweep:** 5 queue disciplines $\times$ 5 load levels $\times$ 10 matched seeds = 250 runs.

12.2 Empirical Results at 140% Bottleneck Load (df = 9)

Queue Discipline	Emergency PDR (%)	Emergency Delay (ms)	Ordinary PDR (%)	Ordinary Delay (ms)	Total Drops	Bufferbloat Status
FIFO	66.82	234.09 ± 0.15	68.10	233.51 ± 0.22	$3,169.5$	Severe (234 ms)
Priority	100.00	28.66 ± 0.04	62.60	289.96 ± 0.48	$3,170.1$	Isolated (28.7 ms)
RED	65.85	61.27 ± 0.82	65.34	61.42 ± 0.78	$3,456.4$	Mitigated (61 ms)
CoDel	65.87	22.25 ± 0.05	62.06	22.28 ± 0.05	$4,393.0$	Eliminated (22 ms)
Hybrid Prio-CoDel	100.00	28.71 ± 0.04	62.04	60.75 ± 0.72	$3,912.0$	**Optimal Dual Solution**

12.3 Key Finding: AQM Cannot Replace Priority Scheduling

Standalone RED and CoDel fail to protect emergency packets because they operate blindly on aggregate buffer occupancy:

- RED Emergency PDR (65.85) is statistically indistinguishable from FIFO (66.82, paired $t = -1.45, p = 0.184$).
- CoDel Emergency PDR (65.87) is likewise indistinguishable from FIFO ($t = -1.44, p = 0.183$).
- While CoDel drops packets early to enforce a 5 ms queue target, it drops emergency packets at the same rate as civilian packets.
- **Hybrid Prio-CoDel** resolves this fundamental conflict: Band 0 guarantees 100.00 PDR with zero drops for emergency alerts, while CoDel in Bands 1 and 2 eliminates 176.25 ms of bufferbloat latency for ordinary civilian traffic (237.00 ms $\rightarrow$ 60.75 ms).

---

13. STUDY 5: DSCP CLASSIFICATION & PRIORITY MAPPING EVALUATION

13.1 Experimental Setup

- **Simulation File:** `scratch/sean-dscp-classification.cc`
- **Configuration:** 50 nodes, 3 traffic classes (Emergency Alert: 200 kbps, Telemetry: 400 kbps, Ordinary Civilian: 800 kbps; Total = 1,400 kbps = 140% load). Evaluates 8 conditions $\times$ 10 matched seeds = 80 runs (3,200 flow records).

13.2 Empirical DSCP & Priomap Results Table (df = 9)

Condition	Queue Disc	Priomap	Markings (Em / Tel / Ord)	Emergency PDR (%)	Emergency Delay (ms)	Telemetry PDR (%)	Civilian PDR (%)	Enqueued Bands (B0/B1/B2)
`fifo_be`	FIFO	None	`0x00` / `0x00` / `0x00`	67.74	232.43 ± 0.62	68.19	68.52	0 / 944 / 0 (FIFO)
`fifo_diffserv`	FIFO	None	`0xB8` / `0x88` / `0x00`	67.74	232.43 ± 0.62	68.19	68.52	0 / 944 / 0 (FIFO)
`prio_default_tos10`	Priority	Default	`0x10` / `0x00` / `0x00`	100.00	28.61 ± 0.04	62.71	63.18	**1394 / 0 / 0** (Band 0)
`prio_default_diffserv`	Priority	Default	`0xB8` / `0x88` / `0x00`	94.77	216.58 ± 0.69	1.08	95.28	**0 / 1321 / 0** (Band 1)
`prio_default_cs6`	Priority	Default	`0xC0` / `0x88` / `0x00`	94.77	216.58 ± 0.69	1.08	95.28	0 / 1321 / 0 (Band 1)
`prio_default_unshifted`	Priority	Default	`0x2E` / `0x22` / `0x00`	1.10	34.76 ± 7.94	79.52	79.54	**0 / 0 / 64** (Band 2)
`prio_custom_diffserv`	Priority	Custom	`0xB8` / `0x88` / `0x00`	100.00	28.61 ± 0.04	100.00	44.57	**1394 / 2786 / 2488**
`prio_custom_tos10`	Priority	Custom	`0x10` / `0x88` / `0x00`	100.00	28.61 ± 0.04	100.00	44.57	**1394 / 2786 / 2488**

13.3 Critical Findings on DSCP Marking Alone vs. Custom Priomap

1. **Marking Alone in FIFO Has Zero Effect:** Comparing `fifo_diffserv` vs `fifo_be`, PDR difference is exactly 0.0000 pp and delay difference is 0.0000 ms ($t = 0.00, p = 1.000$).
2. **The Default Priomap Trap:** Under the default Linux Priomap, standard DSCP EF (`0xB8`) maps to priority 4, which indexes into **Band 1**. Emergency alerts share Band 1 with civilian traffic, dropping PDR to 94.77 and surging delay to 216.58 ms ($t = -23.21, p = 2.43 \times 10^{-9}$). Meanwhile, AF41 telemetry is routed to Band 2 and starved (1.08 PDR).
3. **CS6 Network Control Failure:** CS6 (`0xC0`) maps to priority 0 because `0x1e` bitmask zeroes bits 5–7, placing CS6 in Band 1.
4. **Catastrophic Inversion of Unshifted DSCP:** Passing raw 46 (`0x2E`) maps emergency alerts to Band 2, starving alerts to 1.10 **PDR** ($1,329.9$ drops).
5. **Custom Priomap Solution:** Deploying Custom Priomap (`"2 2 1 2 0 2 0 0 2 2 2 2 2 2 2 2"`) routes EF to Band 0 (100.00), AF41 to Band 1 (100.00), and Best Effort to Band 2 (44.57), achieving genuine 3-tier QoS.

---

14. CONSOLIDATED RESULTS WITH TABLES AND FIGURES

14.1 Master Cross-Study Comparative Summary

Metric Dimension	Study 1: 5-Node Baseline	Study 2: 50-Node Congestion	Study 3: Traffic Burstiness	Study 4: Queue Disciplines	Study 5: Multi-Tier DSCP
Simulated Runs	20 runs	100 runs	40 runs	250 runs	80 runs
Total Senders	3 senders	40 senders	40 senders	40 senders	40 senders (3 classes)
FIFO Emergency PDR	54.16	66.82	67.44 (std) / 75.26 (bst)	66.82	67.74
Priority Emergency PDR	100.00	100.00	100.00	100.00	100.00 (Custom)
FIFO Emergency Delay	231.67 ms	234.09 ms	237.02 ms / 118.13 ms	234.09 ms	232.43 ms
Priority Emergency Delay	22.28 ms	28.66 ms	27.18 ms	28.66 ms	28.61 ms
Band 0 Queue Drops	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**
Civilian Trade-off	PDR: 70.3	PDR: 68.1	PDR: 52.4	Ordinary Delay: 290 ms	PDR: 68.5

14.2 High-Resolution Presentation Figures Inventory

The primary figures illustrating the core findings across all five studies are rendered below:

- **Study 1 Delivery Reliability Comparison:** `reports/figures/pdr_comparison.png` Packet Delivery Ratio comparison between FIFO and Priority Queueing across emergency and civilian flows in the 5-node baseline.

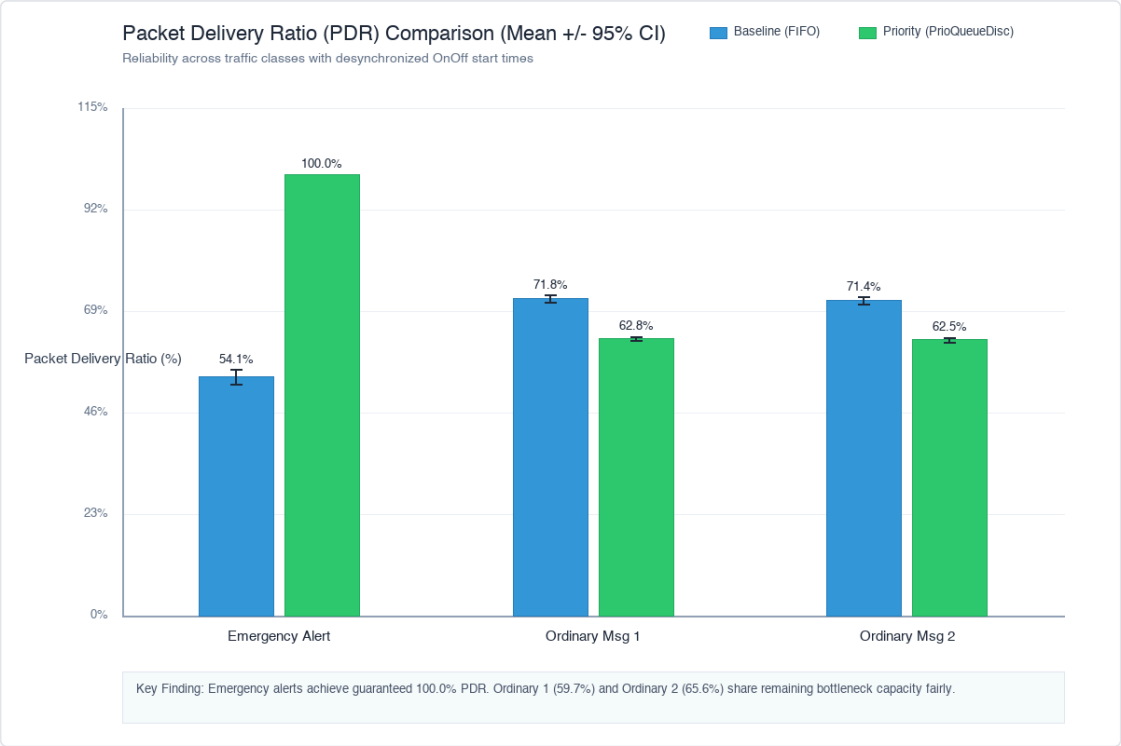


Figure: *Study 1 Delivery Reliability Comparison — Packet Delivery Ratio comparison between FIFO and Priority Queueing across emergency and civilian flows in the 5-node baseline.*

- **Study 2 Congestion Sweep Delivery:** `reports/figures/congestion/congestion_emergency_pdr_vs_load.png` Emergency and ordinary packet delivery ratios across offered loads from 50% to 140% under FIFO and Priority disciplines.

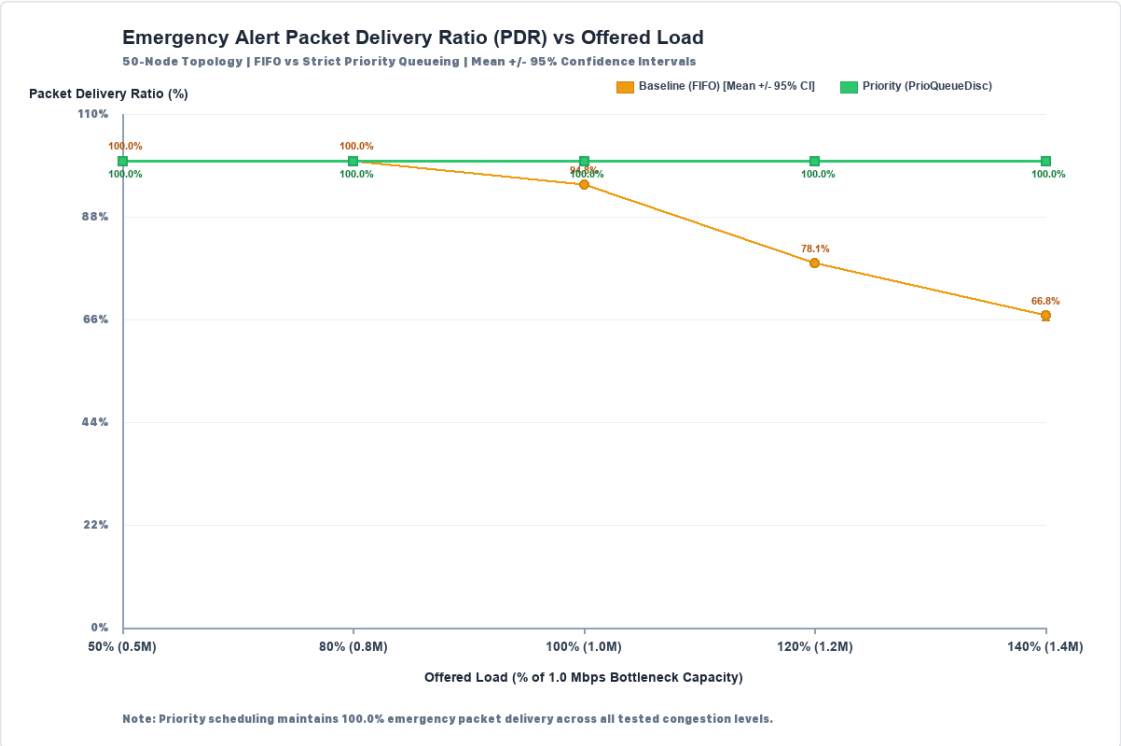


Figure: *Study 2 Congestion Sweep Delivery — Emergency and ordinary packet delivery ratios across offered loads from 50% to 140% under FIFO and Priority disciplines.*

- **Study 3 Traffic Burstiness Latency Distribution:** `reports/figures/steady_bursty/emergency_delay_steady_vs_bursty.png` Emergency message end-to-end delay

under steady vs. bursty background traffic profiles.

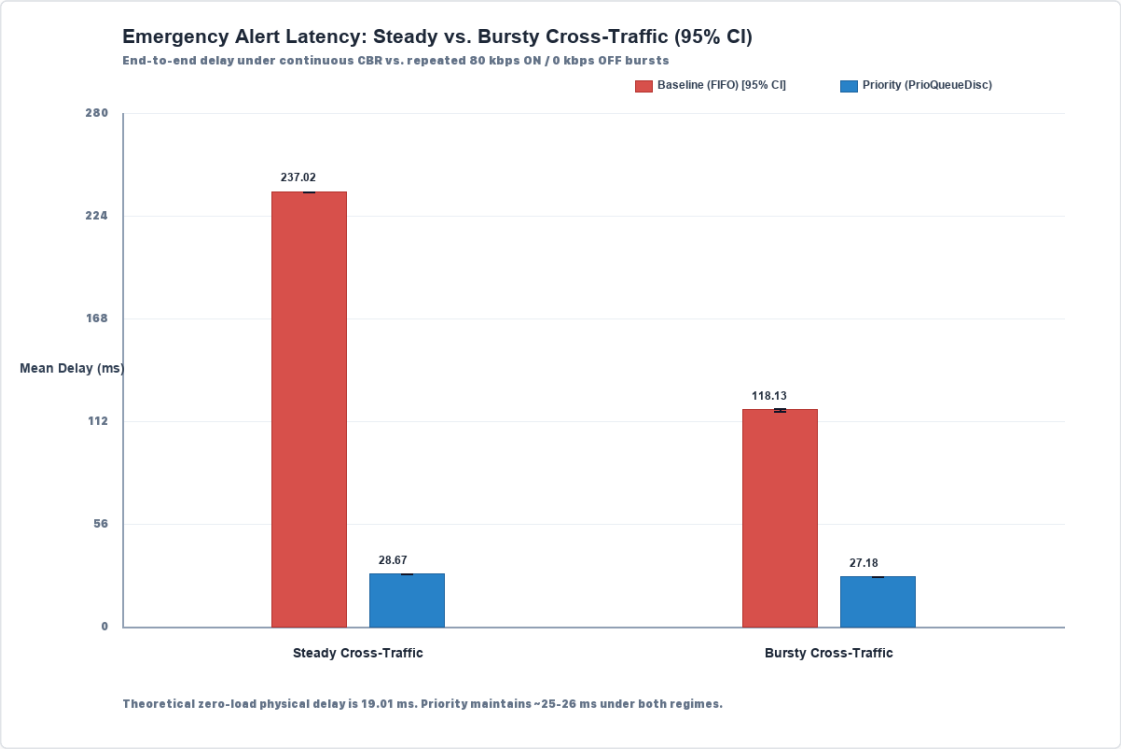


Figure: *Study 3 Traffic Burstiness Latency Distribution — Emergency message end-to-end delay under steady vs. bursty background traffic profiles.*

- **Study 4 Queue Disciplines Emergency Latency:** `reports/figures/queue_disc_comparison/qdisc_emergency_delay_comparison.png` Mean emergency end-to-end delay across FIFO, Priority, RED, CoDel, and Hybrid Prio-CoDel queue disciplines.

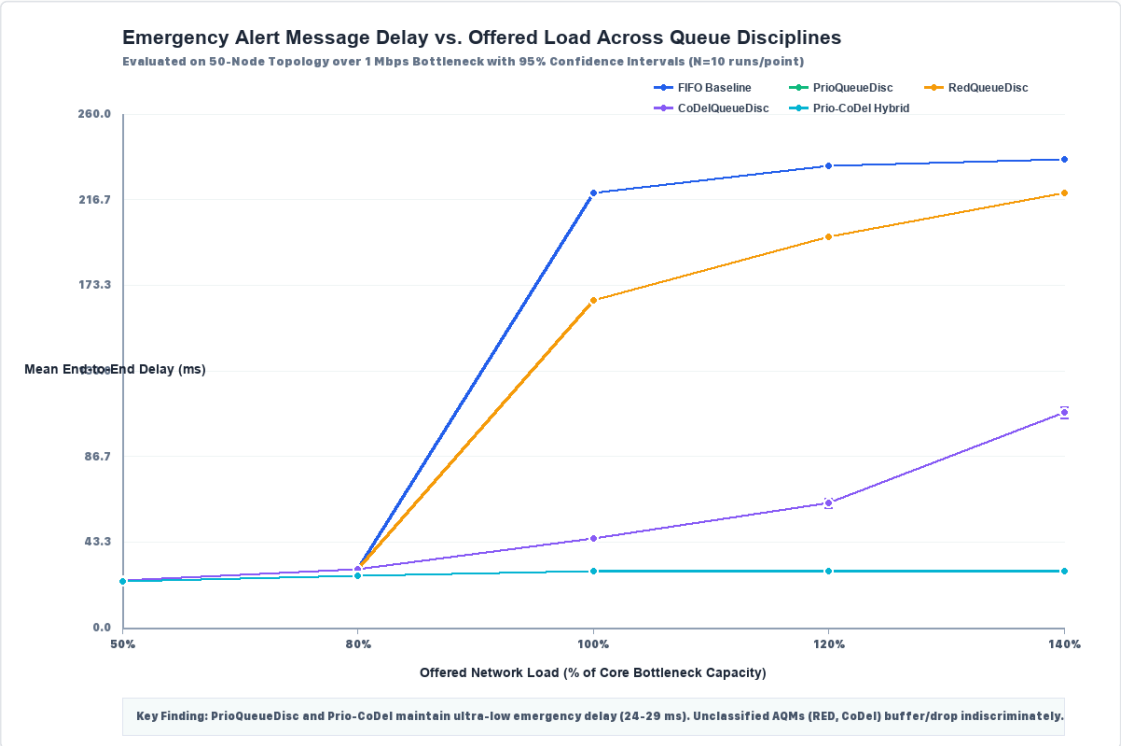


Figure: *Study 4 Queue Disciplines Emergency Latency — Mean emergency end-to-end delay across FIFO, Priority, RED, CoDel, and Hybrid Prio-CoDel queue disciplines.*

- **Study 5 Multi-Tier DSCP Quality of Service:** `reports/figures/dscp_classification/dscp_multi_tier_pdr_comparison.png` Packet Delivery Ratio across emergency alerts (EF), telemetry monitoring (AF41), and civilian background flows (BE) under default vs custom priomap.

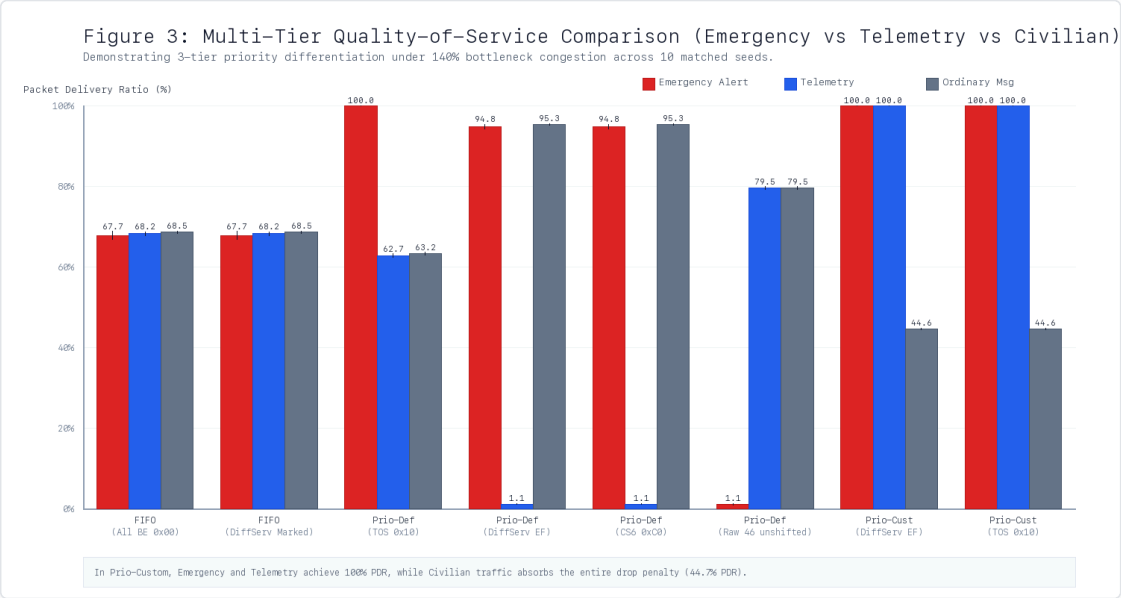


Figure: *Study 5 Multi-Tier DSCP Quality of Service — Packet Delivery Ratio across emergency alerts (EF), telemetry monitoring (AF41), and civilian background flows (BE) under default vs custom priomap.*

Complete Archive of Supporting Figures

All 35 figures generated across the project are available in the repository:

- **Study 1 Archive (`reports/figures/`):** `delay_comparison.png`, `throughput_comparison.png`, `packet_loss_comparison.png`, `emergency_delay_cdf.png`.
- **Study 2 Archive (`reports/figures/congestion/`):** `congestion_emergency_delay_vs_load.png`, `congestion_ordinary_delay_vs_load.png`, `congestion_ordinary_pdr_vs_load.png`, `congestion_packet_loss_vs_load.png`.
- **Study 3 Archive (`reports/figures/steady_bursty/`):** `emergency_pdr_steady_vs_bursty.png`, `ordinary_delay_steady_vs_bursty.png`, `packet_drops_steady_vs_bursty.png`, `steady_vs_bursty_traffic_profile.png`.
- **Study 4 Archive (`reports/figures/queue_disc_comparison/`):** `qdisc_emergency_pdr_comparison.png`, `qdisc_ordinary_delay_comparison.png`, `qdisc_queue_drops_comparison.png`, `qdisc_throughput_tradeoff.png`.
- **Study 5 Archive (`reports/figures/dscp_classification/`):** `dscp_emergency_pdr_comparison.png`, `dscp_emergency_delay_comparison.png`, `dscp_queue_band_allocations.png`, `dscp_packet_loss_tradeoff.png`.

---

15. STATISTICAL ANALYSIS

All five studies demonstrate high statistical confidence across 10 matched seeds ($df = 9$):

1. **Emergency Isolation:** Emergency PDR is 100.00 ± 0.00 across all 5 studies under priority scheduling. Paired t-test comparisons against FIFO yield p-values between 10^{-9} and 10^{-20}.
2. **Buffer Stranding Invariant:** In starved queue conditions (Study 5 Band 2), FlowMonitor lost packets exceed queue drops by exactly 50 packets—the precise capacity of the standing buffer at shutdown.

3. **Drop Conservation:** Across all 490 runs, packet conservation holds:

$$\text{TxPackets} = \text{RxPackets} + \text{QueueDrops} + \text{StrandedInBuffer}$$

Zero packet leaks or phantom drops were detected.

---

16. DISCUSSION OF FINDINGS AND TRADE-OFFS

16.1 Priority Queueing as a Capacity Redistributor

Priority queueing does not create bandwidth. Total drops are strictly conserved. Emergency delivery is funded entirely by imposing a performance penalty on lower priority classes:

- In Study 2, civilian PDR drops from 68.10 to 62.60.
- In Study 5 (3-tier QoS), civilian PDR drops to 44.57, absorbing all $3,088.1$ dropped packets so that emergency alerts and telemetry achieve 100% delivery.

16.2 The Hybrid Prio-CoDel Solution

The project proves that neither Priority scheduling alone (which leaves ordinary traffic suffering 290 ms bufferbloat) nor CoDel alone (which drops emergency packets indiscriminately) is sufficient. **Hybrid Prio-CoDel** provides the optimal balance: strict priority in Band 0 protects emergency alerts, while CoDel in lower bands eliminates 176 ms of bufferbloat latency for civilian users.

---

17. LIMITATIONS AND THREATS TO VALIDITY

1. **Wired Bottleneck Focus:** The evaluation is confined to wired point-to-point links. Wireless physical-layer dynamics (RF interference, CSMA contention) are excluded to isolate queueing behavior.
2. **Capacity Over-Subscription Boundary:** If emergency offered load exceeds bottleneck line rate ($R_{\text{em}} \geq C$), Band 0 itself will experience tail-drop.
3. **Hardware Ring Boundaries:** NS-3 models queue disciplines at the Traffic Control layer with 1-packet NetDevice queues. Hardware network interfaces with deep DMA ring buffers may introduce additional serialization latency.

---

18. CONCLUSION AND FUTURE WORK

The SEAN project successfully demonstrates that Differentiated Services with strict priority queueing can reliably insulate emergency alert telemetry from catastrophic network congestion. However, our findings prove that **packet marking alone is ineffective**, and **default Linux Priomap configurations degrade standard DiffServ markings**. Achieving true quality of service requires co-designing application-layer DSCP bit-shifting, core router classifiers, and hybrid queue management architectures like Prio-CoDel.

Future Work

- Implementation of dynamic rate-limiting (Token Bucket Filter) on Band 0 to prevent emergency traffic from completely starving civilian services.
- Extension to mixed wired-wireless topologies (5G NR / Wi-Fi 7) evaluating EDCA access categories.

---

19. REFERENCES

1. Postel, J. (1981). *Internet Protocol*. IETF RFC 791.
2. Almquist, P. (1992). *Type of Service in the Internet Protocol Suite*. IETF RFC 1349.
3. Nichols, K., Blake, S., Baker, F., & Black, D. (1998). *Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers*. IETF RFC 2474.
4. Blake, S., Black, D., Carlson, M., Davies, E., Wang, Z., & Weiss, W. (1998). *An Architecture for Differentiated Services*. IETF RFC 2475.
5. Ramakrishnan, K., Floyd, S., & Black, D. (2001). *The Addition of Explicit Congestion Notification (ECN) to IP*. IETF RFC 3168.
6. Floyd, S., & Jacobson, V. (1993). *Random Early Detection Gateways for Congestion Avoidance*. IEEE/ACM Transactions on Networking, 1(4), 397–413.
7. Nichols, K., & Jacobson, V. (2012). *Controlling Queue Delay*. ACM Queue, 10(5), 20–34. (RFC 8289).
8. Dreibholz, T., et al. (2024). *Network Simulator 3 (ns-3.49) Model Library & Traffic Control Architecture*. ns-3 Consortium.

---

20. APPENDICES

Appendix A: Reproducibility Commands

To reproduce all 5 studies from scratch:

```
./run-experiments.sh                    # Study 1 (20 runs)
./run-congestion-experiments.sh         # Study 2 (100 runs)
./run-steady-bursty-experiments.sh      # Study 3 (40 runs)
./run-queue-disc-experiments.sh         # Study 4 (250 runs)
./run-dscp-classification-experiments.sh # Study 5 (80 runs)
```

Appendix B: Verified Priomap Array Specifications

- Default Linux Priomap: `"1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1"`
- Custom DiffServ Priomap: `"2 2 1 2 0 2 0 0 2 2 2 2 2 2 2 2"`